

TRƯỜNG ĐẠI HỌC KHOA HỌC
KHOA CÔNG NGHỆ THÔNG TIN



TIỂU LUẬN MÔN HỌC
PHÁT TRIỂN ỨNG DỤNG IOT

Đề Tài:

PHÁT TRIỂN HỆ THỐNG CẢNH BÁO
NGẬP NƯỚC VỚI ESP32

Giáo viên hướng dẫn:
Thầy Võ Việt Dũng

Sinh viên thực hiện :
Cao Thọ Hà Tuyên
Nhóm : 2
Lớp: K46-I
Năm học: 2025-2026

Huế, 04/2026

MỤC LỤC

LỜI MỞ ĐẦU	3
CHƯƠNG 1: TỔNG QUAN VỀ CÔNG NGHỆ VÀ THIẾT BỊ.....	3
1.1. Vi điều khiển trung tâm ESP32	3
1.2. Giao thức truyền thông I2C và Màn hình LCD 16x2.....	3
1.3. Cảm biến giả lập (Biến trở - Potentiometer)	3
1.4. Hệ thống cảnh báo tại chỗ (LED và Buzzer).....	4
1.5. Nền tảng Cloud ThingSpeak.....	4
CHƯƠNG 2: THIẾT KẾ HỆ THỐNG	4
2.1. Sơ đồ khối của hệ thống	4
2.2. Sơ đồ kết nối của hệ thống	5
2.3. Lưu đồ thuật toán.....	5
CHƯƠNG 3: THỰC NGHIỆM VÀ KẾT QUẢ.....	5
3.1. Môi trường mô phỏng	5
3.2. Kết quả đạt được:.....	8
3.3. Đánh giá	9
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	10

LỜI MỞ ĐẦU

Lý do chọn đề tài: Biến đổi khí hậu khiến tình trạng ngập lụt tại các đô thị (như Huế) diễn ra thường xuyên. Việc phát triển hệ thống cảnh báo sớm là cần thiết để bảo vệ tài sản và tính mạng.

Mục tiêu: Xây dựng mô hình IoT có khả năng đo mức nước, hiển thị tại chỗ và phát tín hiệu báo động khẩn cấp.

CHƯƠNG 1: TỔNG QUAN VỀ CÔNG NGHỆ VÀ THIẾT BỊ

1.1. Vi điều khiển trung tâm ESP32

Trong dự án này, ESP32 được lựa chọn làm "bộ não" điều khiển thay vì các dòng vi điều khiển khác (như Arduino Uno) vì những lý do sau:

Tích hợp sẵn WiFi: Đây là yếu tố then chốt để thực hiện bài toán IoT, cho phép kết nối trực tiếp với Cloud ThingSpeak mà không cần thêm module ngoài.

Tốc độ xử lý cao: Với nhân kép và tốc độ lên tới 240MHz, ESP32 đảm bảo việc đọc dữ liệu cảm biến và cập nhật màn hình LCD diễn ra mượt mà.

Nhiều chân ADC: Phù hợp để mở rộng thêm nhiều cảm biến đo mực nước hoặc lưu lượng dòng chảy trong tương lai.

1.2. Giao thức truyền thông I2C và Màn hình LCD 16x2

Màn hình LCD 16x2 kết hợp với module chuyển đổi I2C được sử dụng để hiển thị thông tin tại chỗ:

Tiết kiệm tài nguyên: Giao tiếp I2C chỉ sử dụng 2 chân (SDA và SCL), giúp tiết kiệm chân GPIO cho các linh kiện khác như còi báo và LED.

Thực quan hóa: Giúp người dùng quan sát trực tiếp mực nước (%) và trạng thái cảnh báo ngay tại hiện trường mà không cần mở điện thoại hay máy tính.

1.3. Cảm biến giả lập (Biến trở - Potentiometer)

Trong môi trường mô phỏng Wokwi, biến trở là lựa chọn tối ưu nhất để thay thế cảm biến mực nước thực tế:

Giả lập tín hiệu Analog: Biến trở tạo ra sự thay đổi điện áp từ 0V đến 3.3V, tương ứng với mức nước từ 0% đến 100%.

Kiểm thử linh hoạt: Cho phép người thực hiện dễ dàng xoay núm vặn để kiểm tra độ nhạy của ngưỡng báo động (Threshold) và xem dữ liệu thay đổi trên biểu đồ ThingSpeak.

1.4. Hệ thống cảnh báo tại chỗ (LED và Buzzer)

Việc chọn LED và Còi báo là bắt buộc đối với một hệ thống cảnh báo thiên tai:

Còi báo (Buzzer): Sử dụng âm thanh tần số cao để gây chú ý tức thì ngay cả khi người dân đang ngủ hoặc ở xa bảng điều khiển.

Đèn LED đa sắc: Giúp phân loại mức độ nguy hiểm bằng thị giác (Xanh: An toàn, Đỏ: Nguy hiểm), giúp việc nhận biết trạng thái hệ thống trở nên nhanh chóng và đơn giản.

1.5. Nền tảng Cloud ThingSpeak

Hệ thống sử dụng ThingSpeak làm nền tảng giám sát từ xa vì:

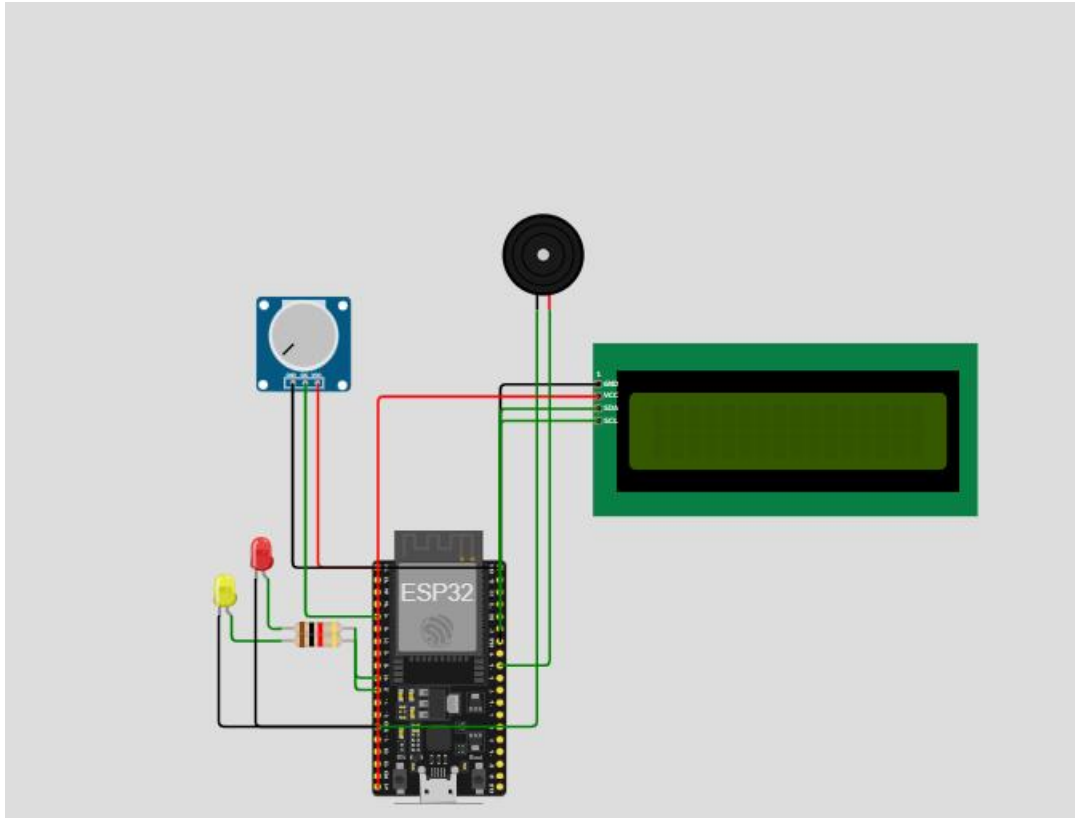
Tính thời gian thực: Cho phép lưu trữ và vẽ biểu đồ dữ liệu mực nước liên tục, giúp theo dõi diễn biến lũ lụt theo biểu đồ thời gian.

Khả năng mở rộng: Dễ dàng tích hợp các thông báo qua Email hoặc gửi tin nhắn về Smartphone khi mực nước vượt ngưỡng an toàn.

CHƯƠNG 2: THIẾT KẾ HỆ THỐNG

2.1. Sơ đồ khối: (Bạn mô tả: Khối cảm biến -> Khối xử lý ESP32 -> Khối đầu ra bao gồm LCD, LED, Buzzer).

2.2. Sơ đồ kết nối (Wiring):



Giải thích chân nối: Cảm biến (chân 34), LCD (SDA-21, SCL-22), Buzzer (chân 18), LED (26, 27).

2.3. Lưu đồ thuật toán:

Khởi động WiFi và LCD.

Đọc giá trị từ cảm biến.

Kiểm tra: Nếu giá trị > ngưỡng (60%) thì hú còi và hiện cảnh báo.

Ngược lại thì hiển thị trạng thái an toàn.

CHƯƠNG 3: THỰC NGHIỆM VÀ KẾT QUẢ

3.1. Môi trường mô phỏng: Sử dụng trình giả lập Wokwi để kiểm chứng thuật toán mà không cần phần cứng thực tế.

- Mã nguồn:

```
#include <WiFi.h>
#include <LiquidCrystal_I2C.h>
#include "ThingSpeak.h"
```

```

const char* ssid = "Wokwi-GUEST";
const char* password = "";

unsigned long myChannelNumber = 3336404;
const char * myWriteAPIKey = "PGZyFCAWUFpyGOZR";

WiFiClient client;
LiquidCrystal_I2C lcd(0x27, 16, 2);

const int POT_PIN = 34;
const int LED_RED = 26;
const int LED_YELLOW = 27;
const int BUZZER_PIN = 18;
const int THRESHOLD = 2500;

unsigned long lastTime = 0;
unsigned long timerDelay = 15000;
void setup() {
  Serial.begin(115200);
  pinMode(LED_RED, OUTPUT);
  pinMode(LED_YELLOW, OUTPUT);
  pinMode(BUZZER_PIN, OUTPUT);

  lcd.init();
  lcd.backlight();
  lcd.print("DANG KET NOI...");

  WiFi.begin(ssid, password, 6);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");

  ThingSpeak.begin(client);
  lcd.clear();
}

```

```

void loop() {
  int value = analogRead(POT_PIN);
  int percent = map(value, 0, 4095, 0, 100);

  lcd.setCursor(0, 0);
  lcd.print("Muc nuoc: ");
  lcd.print(percent);
  lcd.print("% ");

  // Logic báo động
  if (value > THRESHOLD) {
    digitalWrite(LED_RED, HIGH);
    digitalWrite(LED_YELLOW, LOW);
    lcd.setCursor(0, 1);
    lcd.print("NGUY HIEM: NGAP!");
    tone(BUZZER_PIN, 1500);
    delay(150);
    tone(BUZZER_PIN, 1000);
    delay(150);
  } else {
    noTone(BUZZER_PIN);
    digitalWrite(LED_RED, LOW);
    digitalWrite(LED_YELLOW, HIGH);
    lcd.setCursor(0, 1);
    lcd.print("Trang thai: OK ");
  }

  if ((millis() - lastTime) > timerDelay) {
    int x = ThingSpeak.writeField(myChannelNumber, 1, percent, myWriteAPIKey);

    if (x == 200) {
      Serial.println("Channel update successful.");
    }
    else {
      Serial.println("Problem updating channel. HTTP error code " + String(x));
    }
    lastTime = millis();
  }
}

```

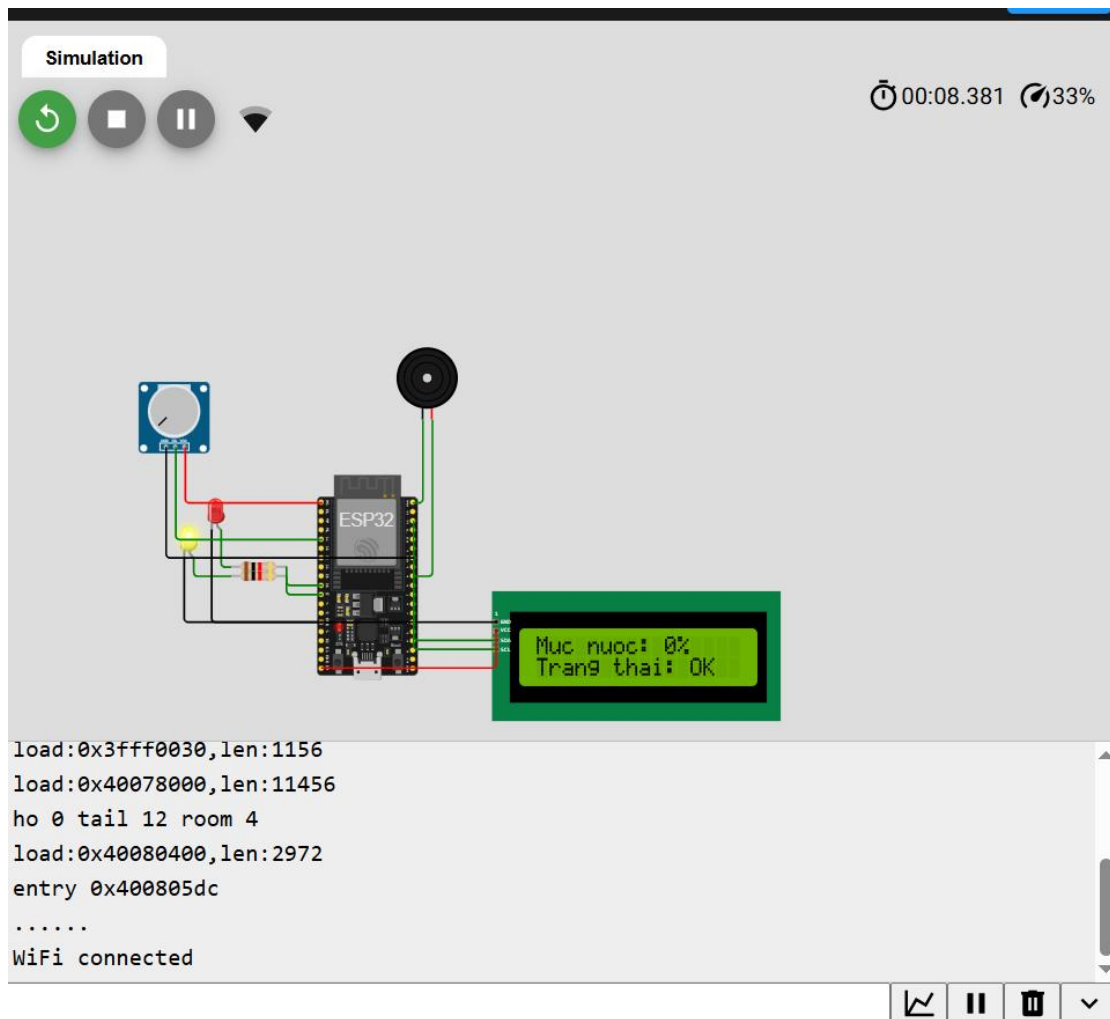
```

}
}

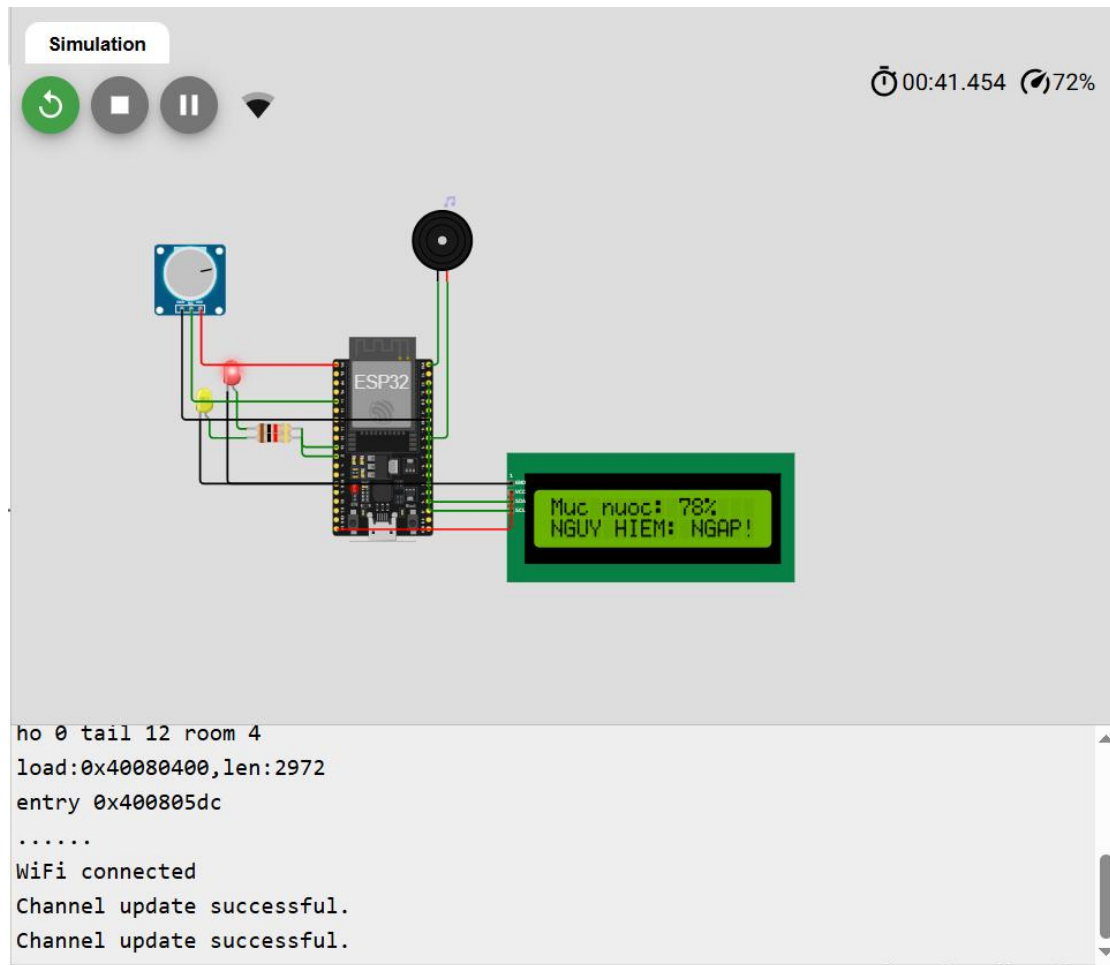
```

3.2. Kết quả đạt được:

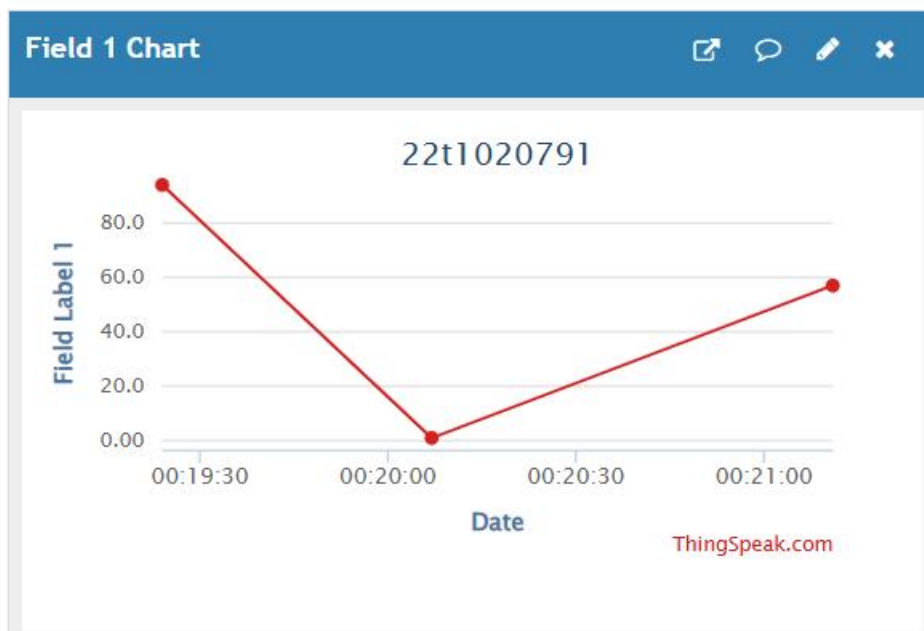
Trạng thái bình thường:



Trạng thái khẩn cấp:



Kết quả ở Thingspeak:



3.3. Đánh giá: Hệ thống phản hồi nhanh, âm thanh báo động rõ ràng và thông báo được đẩy lên Serial Monitor.

Ghi chú: "Hệ thống đã được kiểm chứng tính đúng đắn qua mã nguồn, tuy nhiên do hạn chế về hạ tầng máy chủ mô phỏng tại thời điểm thực hiện, các kết quả được ghi nhận qua Serial Monitor."

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết luận: Đã hoàn thành mô hình cảnh báo ngập nước cơ bản sử dụng ESP32.

Hướng phát triển:

Kết nối với ứng dụng điện thoại (như Blynk hoặc Telegram) để nhận thông báo từ xa.

Tích hợp thêm Relay để tự động ngắt điện khi có ngập lụt nhằm đảm bảo an toàn.